

Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia de
Telecomunicações e Informática
Escola de Engenharia

Relatório Final do Projeto de Redes Móveis

Simulação CDMA

Trabalho elaborado por:

Luís Pedro Cerqueira Baptista, a105419

Bernardo Gualdino Salgado, a103664

Luís Filipe da Silva Marques, a103674

2024/2025

Índice

1 Introdução	4
2 Arquitetura do Sistema	4
3 Resultados	6
3.1 Demo 1 - Transmissor + Canal	6
3.2 Demo 2 - Transmissor + Canal + Receptor (sem sincronização).....	7
3.3 Demo 3 — Sincronização em canal CDMA (1 utilizador)	9
3.4 Demo 4 — Sincronização multi-utilizador (offsets aleatórios)	10
3.4.1 Estratégia de Sincronização Implementada	13
4 Resultados de Aprendizagem.....	14
5 Conclusão	15
6 Referências	15

Índice de Imagens

Figura 1 - Diagrama de Blocos	5
Figura 2 - Demo 1 resultados.....	7
Figura 3 - BER vs SNR (3 users) da Demo 2.....	8
Figura 4 - BER vs SNR (7 users) da Demo 2.....	8
Figura 5 - BER vs SNR Demo 3.....	10
Figura 6 - BER vs SNR (3 utilizadores) offset fixo	10
Figura 7 - BER vs SNR (7 utilizadores) offset fixo.....	11
Figura 8 - BER vs SNR (3 utilizadores) offset fixo + coeficiente de atenuação fixos .	11
Figura 9 - BER vs SNR (3 utilizadores) offset aleatório	12
Figura 10 - BER vs SNR (5 utilizadores) offset aleatório	12

1 Introdução

A crescente procura por serviços de comunicações móveis com altas taxas de transmissão e suporte a múltiplos utilizadores tornou essencial o estudo e implementação de técnicas eficientes de acesso ao meio e de partilha do espectro. Entre estas técnicas destaca-se o CDMA (Code Division Multiple Access), amplamente utilizado em redes móveis, pela sua capacidade de permitir a partilha simultânea do canal de rádio por vários utilizadores, utilizando códigos de espalhamento ortogonais.

Este projeto, realizado no âmbito da unidade curricular de Redes Móveis da Universidade do Minho, teve como objetivo principal a simulação de um sistema CDMA multi-utilizador, abordando desde a geração dos sinais até à análise detalhada da sua receção e desempenho em diferentes condições de canal. Foram desenvolvidos, em Python, todos os módulos essenciais de um sistema CDMA: transmissores, canal (com adição de ruído e atenuação), recetor, medidor de BER e simulador.

O trabalho foi estruturado por etapas, cada uma correspondendo a uma demonstração (Demo), com objetivos de complexidade crescente:

- Desde a simples geração e transmissão de sinais (Demo 1),
- Passando pela recuperação das mensagens sem e com sincronização (Demos 2 e 3, respetivamente),
- Até à simulação de cenários realistas, onde cada utilizador entra no canal em instantes diferentes e a recuperação das suas mensagens é feita com sincronização (Demo 4).

Ao longo do desenvolvimento, foi possível observar na prática o impacto do ruído, da interferência multi-utilizador e da sincronização na fiabilidade do sistema, consolidando conhecimentos teóricos essenciais das comunicações digitais e das redes móveis.

2 Arquitetura do Sistema

O sistema desenvolvido é composto por cinco módulos principais representados na Figura 1:

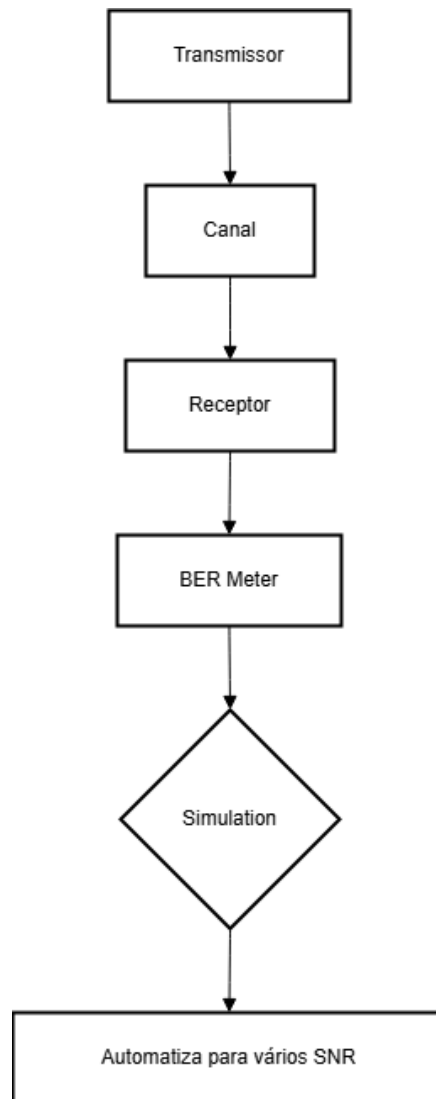


Figura 1 - Diagrama de Blocos

1. Transmissores

- Geram sinais CDMA para cada utilizador, com mensagens aleatórias, códigos de espalhamento ortogonais (Walsh), inclusão de campo de sincronização (a partir da Demo 3), fator de espalhamento e amostragem configuráveis.
- Guardam o sinal transmitido, bits, código e parâmetros em ficheiro.

2. Canal

- Combina os sinais de todos os utilizadores.
- Aplica fatores de atenuação (iguais ou aleatórios).
- Adiciona ruído AWGN (ruído gaussiano branco aditivo) com SNR configurável.

- Opcionalmente (Demo 4), cada utilizador pode entrar com um offset temporal aleatório.

3. Recetor

- Recupera as mensagens originais a partir do sinal combinado.
- Utiliza o código de cada utilizador para “despreading”.
- Desde a Demo 3, faz auto-sincronização (procura o início da mensagem usando o padrão de sincronização).
- Na Demo 4, suporta desalinhamento temporal (offsets diferentes) para cada utilizador.

4. BER Meter

- Compara as mensagens originais e recuperadas.
- Calcula a taxa de erro de bits (BER) excluindo o campo de sincronização.
- Guarda os resultados em ficheiro e exhibe no terminal.

5. Simulation

- Controla o funcionamento do sistema para vários valores de SNR.
- Gera automaticamente tudo em memória, recupera as mensagens e calcula o BER.
- Produz gráficos de BER vs SNR para análise de desempenho.

3 Resultados

3.1 Demo 1 - Transmissor + Canal

Objetivo: Gerar e transmitir um sinal CDMA para um único utilizador, adicionar ruído e observar o efeito no sinal.

O que foi feito:

- Geração de mensagem binária aleatória.
- Cálculo e aplicação do código de espalhamento (Walsh).
- Amostragem do sinal.
- Canal adiciona ruído AWGN (SNR configurável).

Os resultados observados representam-se na figura 2:

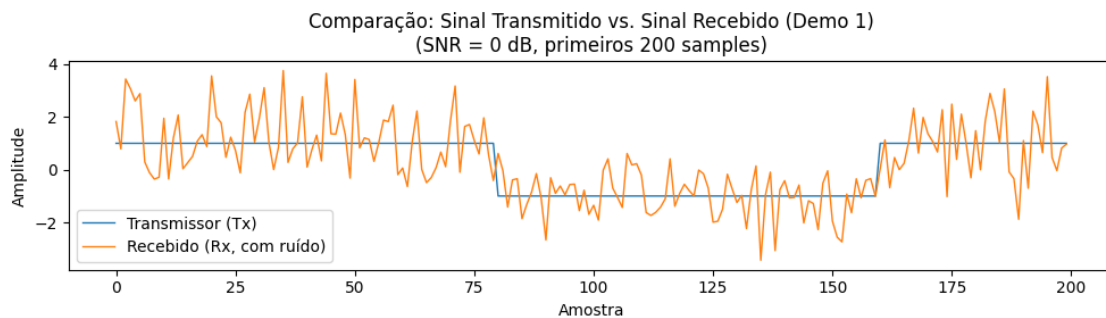


Figura 2 - Demo 1 resultados

Parâmetros usados:

- message_length = 200 # Número de bits
- spreading_factor = 16
- sampling_rate = 5
- SNR_dB = 0

- Visualização do sinal transmitido e do sinal ruidoso no canal.
- Não é possível calcular BER porque não há receptor nesta demo.

Lições aprendidas:

- Percebeu-se a importância da qualidade do canal e do ruído na forma do sinal recebido.
- Fundamentos da codificação e espalhamento de espectro.

3.2 Demo 2 - Transmissor + Canal + Receptor (sem sincronização)

Objetivo: Testar o sistema CDMA para múltiplos utilizadores (tipicamente 3, no máximo até 7 para um spreading factor = 16), analisando o impacto da interferência e do ruído no BER, sem sincronização (o recetor assume início correto).

O que foi feito:

- Geração dos sinais para vários utilizadores.
- Combinação dos sinais no canal com fatores de atenuação.
- Adição de ruído AWGN (SNR variável).
- Receptor aplica o despreading a partir de um offset fixo.
- Cálculo do BER.

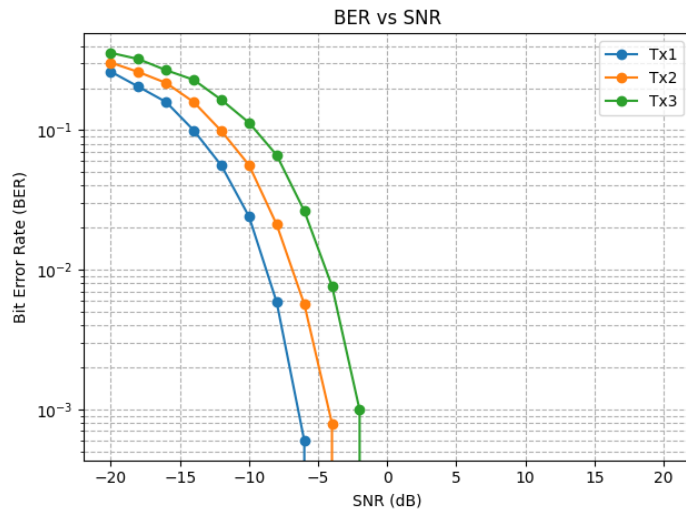


Figura 3 - BER vs SNR (3 users) da Demo 2

Parâmetros usados:

- code_index=0,
- output_filename="transmitted_cdma_signal.txt",
- message_length=10064,
- walsh_order=16,
- spreading_factor=16,
- sampling_rate=5,
- SNR_START_DB = -20
- SNR_END_DB = 20
- SNR_STEP_DB = 2
- NUM_TRIALS = 1
- NUM_TRANSMITTERS = 3
- ATTENUATION_COEFFICIENTS = np.array([1.0, 0.8, 0.6])

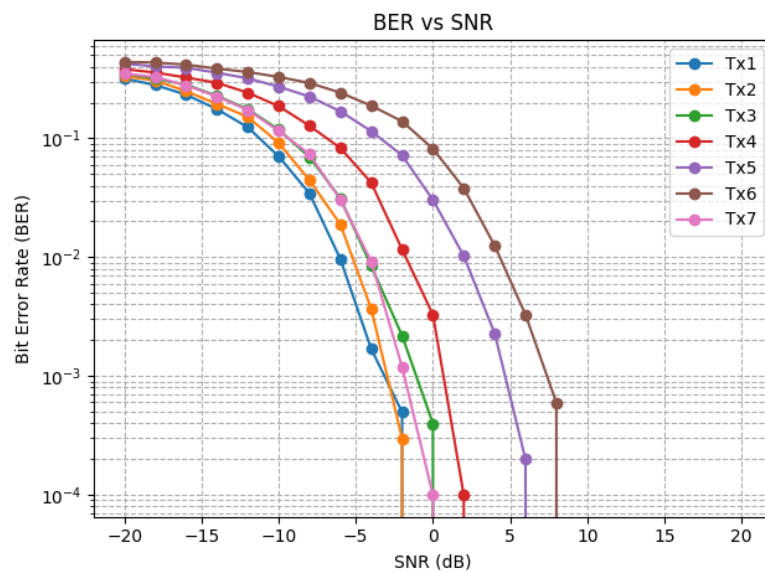


Figura 4 - BER vs SNR (7 users) da Demo 2

Parâmetros usados:

- code_index=0,
- output_filename="transmitted_cdma_signal.txt",
- message_length=10064,
- walsh_order=16,
- spreading_factor=16,
- sampling_rate=5,
- SNR_START_DB = -20
- SNR_END_DB = 20
- SNR_STEP_DB = 2
- NUM_TRIALS = 1
- NUM_TRANSMITTERS = 7
- ATTENUATION_COEFFICIENTS = np.array([1.0, 0.9, 0.8, 0.6, 0.4, 0.3, 0.8])

Resultados observados:

- Gráfico BER vs SNR para diferentes utilizadores.
- Para SNR baixos, BER elevado devido ao ruído e interferência multiutilizador.
- Para SNR altos, BER reduz drasticamente, principalmente quando há poucos utilizadores.

Lições aprendidas:

- O aumento do número de utilizadores aumenta a interferência, degradando o desempenho.
- A ausência de sincronização limita a robustez do sistema.

3.3 Demo 3 — Sincronização em canal CDMA (1 utilizador)

Objetivo: Introduzir o campo de sincronização (sync pattern) na mensagem transmitida, e modificar o receptor para encontrar automaticamente o início do payload, garantindo $BER = 0$ em SNR altos mesmo com ruído.

O que foi feito:

- Transmissor insere campo de sincronização (padrão conhecido).
- Receptor procura o padrão de sync no sinal recebido (correlação).
- Só depois começa a despreading para recuperar o payload.
- Cálculo do BER (ignorando o campo de sync).

Resultados observados:

- Para SNR altos, o BER atinge zero — confirmação da sincronização correta.
- Para SNR baixos, BER ainda sobe devido ao ruído.

Lições aprendidas:

- A importância do campo de sincronização para deteção correta do início do payload.
- Conceito de correlação para auto-sincronização.

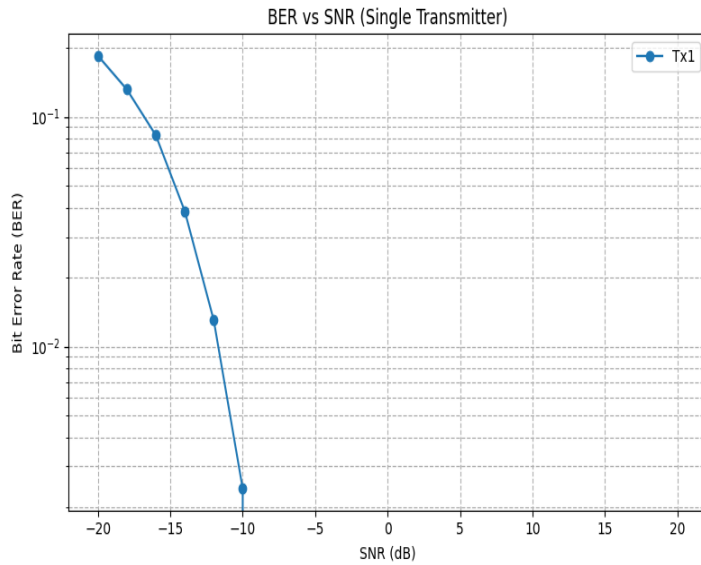


Figura 5 - BER vs SNR Demo 3

Parâmetros usados:

- MESSAGE_LENGTH = 10064
- WALSH_ORDER = 16
- CODE_INDEX = 1
- SPREADING_FACTOR = 16
- SAMPLING_RATE = 5
- ZEROS_LENGTH = 50
- SYNC_PATTERN = np.tile([1,0,1,0,1,1,0,0], 8)
- SYNC_PREAMBLE_LENGTH = len(SYNC_PATTERN)
- SNR_START_DB = -20
- SNR_END_DB = 20
- SNR_STEP_DB = 2
- NUM_TRIALS = 1
- ATTENUATION = 1
- SYNC_CORR_THRESHOLD = 0.5

3.4 Demo 4 — Sincronização multi-utilizador (offsets aleatórios)

Objetivo: Atualizar o recetor de forma a fazer sincronismo multi-utilizador tornando o sistema robusto a desalinhamentos temporais e simulando utilizadores que entram no canal em tempos diferentes (offsets aleatórios).

Antes desta simulação, testamos o mesmo objetivo para offsets fixos onde todos entram ao mesmo tempo no canal, de forma a comparar com a Demo 2 o facto de termos multi-utilizador com (Demo 4) e sem (Demo 2) sincronização.

Deste modo nas Figuras 5 e 6 apresentamos a simulação para 3 e 7 utilizadores respetivamente, tal como nas simulações da Demo 2 de forma a visualizar essa comparação.

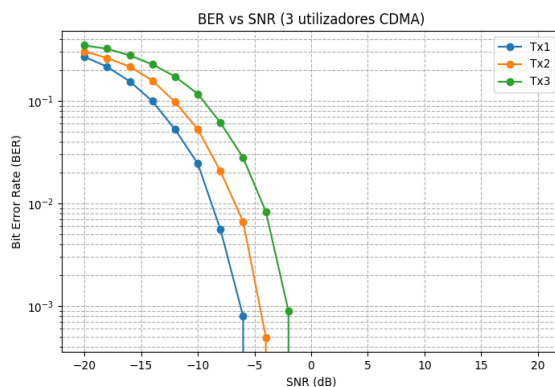


Figura 6 - BER vs SNR (3 utilizadores) offset fixo

Parâmetros usados:

- MESSAGE_LENGTH = 10064
- WALSH_ORDER = 16
- SPREADING_FACTOR = 16
- SAMPLING_RATE = 5
- LEADING_ZEROS = 50
- TX_COUNT = 3
- RUNS_PER_SNR = 1
- SNR_RANGE = np.arange(-20, 21, 2)
- CODE_IDX = generate_code_idx(n_tx, WALSH_ORDER, step=3)
- attenuation_coeffs=np.array([1.0, 0.8, 0.6])

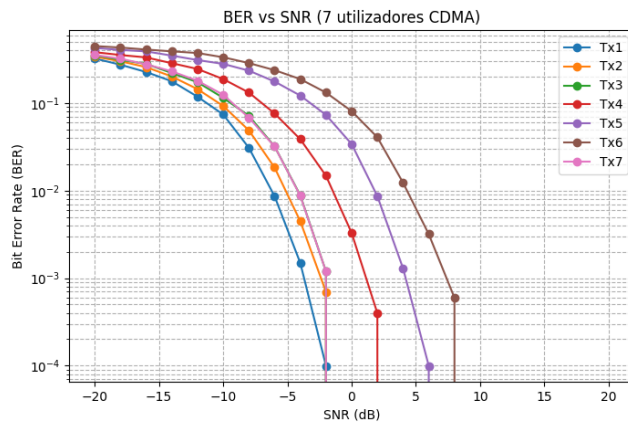


Figura 7 - BER vs SNR (7 utilizadores) offset fixo

Parâmetros usados:

- MESSAGE_LENGTH = 10064
- WALSH_ORDER = 16
- SPREADING_FACTOR = 16
- SAMPLING_RATE = 5
- LEADING_ZEROS = 50
- TX_COUNT = 7
- RUNS_PER_SNR = 1
- SNR_RANGE = np.arange(-20, 21, 2)
- CODE_IDX = generate_code_idx(n_tx, WALSH_ORDER, step=3)
- attenuation_coeffs=np.array ([1.0, 0.9, 0.8, 0.6, 0.4, 0.3, 0.8])

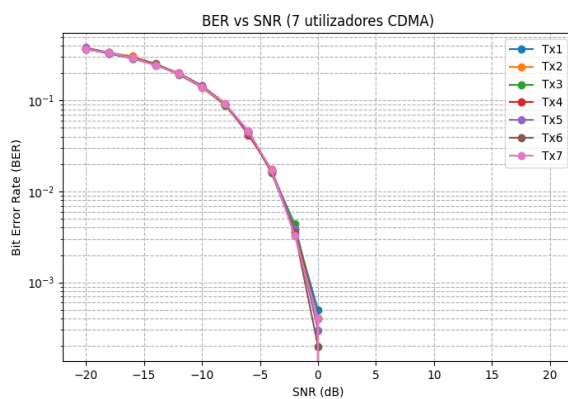


Figura 8 - BER vs SNR (3 utilizadores) offset fixo + coeficiente de atenuação fixos

Parâmetros usados:

- MESSAGE_LENGTH = 10064
- WALSH_ORDER = 16
- SPREADING_FACTOR = 16
- SAMPLING_RATE = 5
- LEADING_ZEROS = 50
- TX_COUNT = 7
- RUNS_PER_SNR = 1
- SNR_RANGE = np.arange(-20, 21, 2)
- CODE_IDX = generate_code_idx(n_tx, WALSH_ORDER, step=3)
- attenuation_coeffs=np.ones(TX_COUNT)

Após simularmos utilizadores que entram no canal em tempos iguais (offset fixos), elevamos ainda mais a fasquia e testamos um cenário real onde utilizadores entram no canal em tempos diferentes (offset aleatório).

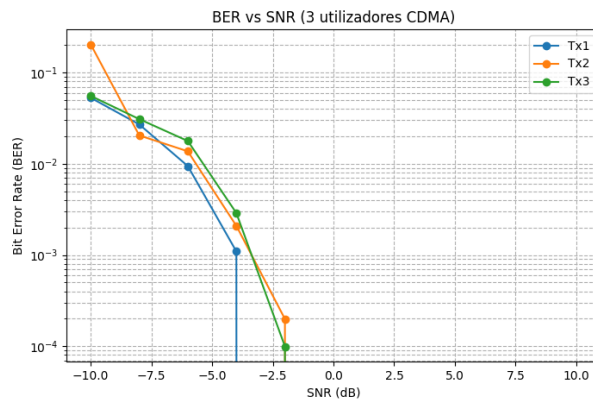


Figura 9 - BER vs SNR (3 utilizadores) offset aleatório

Parâmetros usados:

- MESSAGE_LENGTH = 10064
- WALSH_ORDER = 16
- SPREADING_FACTOR = 16
- SAMPLING_RATE = 5
- LEADING_ZEROS = 50
- TX_COUNT = 3
- max_offset = 3000
- RUNS_PER_SNR = 1
- SNR_RANGE = np.arange(-10, 21, 2)
- CODE_IDX = generate_code_idx(n_tx, WALSH_ORDER, step=3)
- attenuation_coeffs=np.ones(TX_COUNT)

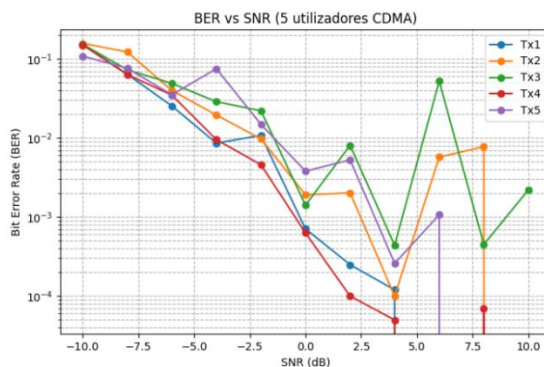


Figura 10 - BER vs SNR (5 utilizadores) offset aleatório

Parâmetros usados:

- MESSAGE_LENGTH = 10064
- WALSH_ORDER = 16
- SPREADING_FACTOR = 16
- SAMPLING_RATE = 5
- LEADING_ZEROS = 50
- TX_COUNT = 5
- max_offset = 3000
- RUNS_PER_SNR = 1
- SNR_RANGE = np.arange(-10, 21, 2)
- CODE_IDX = generate_code_idx(n_tx, WALSH_ORDER, step=3)
- attenuation_coeffs=np.ones(TX_COUNT)

O que foi feito:

- Cada transmissor é atrasado de um valor aleatório antes de ser combinado no canal.
- O receptor procura o padrão de sincronização individualmente para cada utilizador.
- Recuperação do payload para cada utilizador, independentemente do offset.

- Cálculo do BER.

Resultados observados:

- Gráfico BER vs SNR: Todos os utilizadores recuperados corretamente para SNR elevados, mesmo com offsets aleatórios.
- BER semelhante para todos, mostrando robustez do sistema.

Lições aprendidas:

- O sistema é robusto a desalinhamentos temporais se o campo de sincronização for bem implementado.
- Simulação realista de sistemas móveis onde utilizadores podem entrar/sair em momentos diferentes.

3.4.1 Estratégia de Sincronização Implementada

No cenário mais realista da Demo 4, cada utilizador pode iniciar a transmissão em momentos distintos, resultando em offsets temporais aleatórios no sinal combinado. Este facto coloca um desafio adicional ao recetor: como identificar automaticamente o início da mensagem de cada utilizador sem qualquer informação prévia de alinhamento temporal. A estratégia implementada baseou-se nos seguintes passos:

1. Inserção de padrão de sincronização (sync pattern):
Cada transmissor insere, no início da sua mensagem, um padrão de bits conhecido por todos, designado de sync pattern (exemplo: [1,0,1,1,0,1,...]). Este padrão é suficientemente longo e único para ser facilmente detetável mesmo com ruído.
2. Geração do padrão de sincronização local:
Para cada utilizador, o recetor gera localmente o sync pattern, espalha-o com o código de espalhamento (Walsh) desse utilizador e aplica a mesma taxa de oversampling usada na transmissão.
3. Procura por correlação:
O recetor desliza esse padrão sincronizado ao longo do sinal combinado, calculando a correlação (produto interno) em todas as posições possíveis. O máximo da correlação indica a posição mais provável onde o padrão de sincronização desse utilizador aparece.
4. Despreading a partir do pico:
Após detetar a posição do pico, o recetor começa a aplicar o despreading para recuperar o payload (mensagem útil) desse utilizador a partir desse ponto.

Vantagens desta abordagem:

- Permite a correta recuperação da mensagem de cada utilizador, mesmo que entrem no canal em tempos diferentes.
- É robusta ao ruído e à interferência de outros utilizadores, desde que o padrão de sincronização seja suficientemente forte.
- Não depende de sincronismo global ou de qualquer coordenação central, sendo adequada a sistemas reais com acesso assíncrono.

Esta abordagem demonstrou, nos resultados experimentais, ser altamente eficaz, com recuperação perfeita das mensagens para SNR elevados, mesmo com offsets aleatórios aplicados a cada utilizador.

Exemplo de output para 3 utilizadores com offset aleatório em 0dB:

=== SNR 0 dB ===

CODE_IDX usados: [1, 4, 7]

Offsets aplicados (samples): [1323 1859 171]

User 1: start=1573

User 1: len(bits)=10064

User 2: start=2109

User 2: len(bits)=10064

User 3: start=421

User 3: len(bits)=10064

BER = 0.0000%

Como podemos observar toda a mensagem de 10064 bits é recuperada.

4 Resultados de Aprendizagem

- Compreendemos o funcionamento do CDMA e os princípios do espalhamento de espectro.
- Aprendemos a importância dos códigos ortogonais (Walsh) para separar utilizadores e reduzir interferência.
- Percebemos o impacto da sincronização no desempenho do sistema.
- Tornamos-mos proficiente em simulação, análise de desempenho e automação de testes em Python.
- Adquirimos competências em interpretação de resultados práticos e adaptação de sistemas a cenários mais realistas.

5 Conclusão

O desenvolvimento e simulação de um sistema CDMA multi-utilizador permitiu consolidar, de forma prática, os conceitos fundamentais das comunicações móveis e do acesso múltiplo por divisão de código. A implementação modular dos transmissores, canal, receptor, medidor de BER e simulação possibilitou uma análise detalhada das várias fases do sistema, bem como a avaliação do seu desempenho sob diferentes cenários. Foi possível comprovar, através dos resultados experimentais, que:

- A sincronização é um fator crítico para garantir a correta recuperação das mensagens, especialmente em ambientes com múltiplos utilizadores e ruído.
- O sistema, quando corretamente sincronizado e parametrizado, atinge taxas de erro de bits próximas de zero para SNR elevados, mesmo em cenários realistas onde os utilizadores transmitem em tempos distintos.
- O aumento do número de utilizadores implica um maior desafio devido à interferência multiutilizador, tornando essencial o uso de códigos ortogonais e uma boa gestão dos recursos do canal.

Além dos objetivos técnicos, este projeto permitiu desenvolver competências transversais em programação científica, análise de resultados, trabalho em equipa e comunicação de ciência. Em suma, este trabalho constituiu uma excelente oportunidade de aprendizagem aplicada, ligando a teoria à prática e preparando-nos para desafios reais no domínio das comunicações móveis.

6 Referências

Notas da cadeira de Redes Móveis (2024/2025)

Adriano J. C. Moreira, “Project description”, Universidade do Minho, 2025